OWNER SETUP GUIDE

Website Products Setup & operations

Start locally. Add your accounts when you are ready.
A practical guide for your first client and the websites that follow.

START HERE

Open a terminal in your main **website-products** folder and run:

```
npm --prefix admin install
npm run dev
```

Open: http://127.0.0.1:3001

You can add clients, websites, individual maintenance prices, communications and follow-up tasks before connecting external services. Defaults are USD and Europe/Skopje for your North Macedonia business.

WHAT YOU WILL SET UP

01 Local workspace, communications and follow-ups

02 Configuration files and API keys

03 Stripe payment tracking and webhooks

04 Website health and Breeze CMS monitoring

05 Cloud database, owner sign-in and hosting

06 New websites, migration, backups and checks

Prepared 29 October 2026. Keys, account setup and production deployment are completed by you using this guide. Dashboard visual review is left to you as requested.

Prepared 29 October 2026. The implementation lives in this workspace. External accounts, API keys, domains and production deployment are configured by you using the steps below. No payment operations are performed by this admin.

1. Start locally now

Use Node.js 22.16 or a newer Node 22 release. Open a terminal in `C:\Users\Dolov\Desktop\solid-code\website-products`.

```
npm --prefix admin install
npm run dev
```

Open **http://127.0.0.1:3001** or **http://localhost:3001**. Breeze development keeps port 3000; the admin uses 3001. Keep the terminal running and stop it with Ctrl+C. Local development is restricted to this computer. It needs no external keys. `npm start` is production mode and requires hosted identity/backend configuration; use `npm run dev` while working locally.

The first API request registers Breeze, with monitoring disabled and no invented contacts, service prices, health results or payments. Your records persist in `admin/.data/platform.sqlite`. Browser refreshes do not erase them.

In the admin:

1. Edit **Settings > Business details** for your name and email. The business is based in North Macedonia, with USD and Europe/Skopje selected by default.

2. Edit **Clients > Breeze** and enter confirmed contact details.

3. Add individual billing/operations people under **Contacts** if needed.

4. Add maintenance under **Payments > Services**. Set each client's own agreed USD price, billing cycle and covered site; there is no shared maintenance price.

5. Use **Communications** for potential and current clients, and **Tasks** for work.

6. **Websites > Editor** opens Breeze's existing separate CMS login.

Marketing, Calls and Orders are removed from the active interface. Any records previously saved in those modules remain in private backups and exports.

Track potential and existing client communications

1. Open **Communications > Add person**. Enter the person's name, business, email and current conversation stage. A potential client does not need a client or website record yet. Link an existing client when appropriate.

2. Open the person and choose **Log email / note**. Record an email you sent, a reply received or a note, with the actual date and a short subject/summary.

3. The **Last email sent** date is calculated from your sent-email entries. Replies and notes do not change it. Older emails can be logged without moving that date backwards. The timeline preserves repeated reminders and project updates.

4. Set the next follow-up date and next action. Due follow-ups appear on Overview, the Communications navigation badge and the Follow-ups due filter. When logging another email, choose the next reminder date or clear the previous reminder.

5. When a potential client orders, choose **Create client**. Their client and primary contact are created once, and the existing conversation/history stays together. Continue recording communications during and after the website work.

Email dates come from entries you log manually. This feature does not read your mailbox or send reminders automatically. **Open email app** opens your own mail application. Conversation times display in Europe/Skopje; entry inputs use the computer/browser's local time.

2. Configuration files

Create your local configuration from the example:

```
Copy-Item admin/.env.example admin/.env.local
```

Do this once; do not overwrite a file after adding your keys. Restart the dev server after environment changes. Do not prefix secret keys with `NEXT_PUBLIC_`.

Variable	Where to set it	Purpose
`ADMIN_ORIGIN`	Admin local/Vercel	Exact admin browser origin, no trailing slash
`ADMIN_ALLOW_LOCAL`	Admin local only	Local development shortcut; production always rejects it
`PLATFORM_API_URL`	Admin/Vercel	Cloud Worker HTTPS origin; leave blank for local SQLite
`PLATFORM_API_TOKEN`	Admin + Worker secret	Same new server credential on both sides, 32+ characters
`CF_ACCESS_TEAM_DOMAIN`	Admin/Vercel	Your `team.cloudflareaccess.com` hostname
`CF_ACCESS_AUD`	Admin/Vercel	Audience tag from your Access application
`ADMIN_ALLOWED_EMAILS`	Admin/Vercel	Comma-separated authorized owner emails
`STRIPE_MODE`	Local admin or Worker	`test` first; `live` when verified
`STRIPE_SECRET_KEY`	Local admin or Worker secret	Restricted Stripe read key, or secret API key
`STRIPE_WEBHOOK_SECRET`	Local admin or Worker secret	Signing secret for the configured Stripe endpoint
`MONITOR_ALLOWED_HOSTS`	Local admin or Worker vars	Exact approved hostnames, comma-separated
`CMS_MONITOR_TOKENS_JSON`	Local admin or Worker secret	Site-ID-to-read-only-token JSON object

Once `PLATFORM_API_URL` is set, the Worker performs Stripe and monitoring work. Configure those secrets on the Worker, not only in Vercel. API keys never belong in client records, `site.config.json`, source control or browser code.

3. Connect Stripe payment tracking

1. In your own Stripe business account, start in test mode. The tracked payments are clients paying you for maintenance, website builds and related services.

2. Create a restricted read API key with access to the Customers, Invoices and Charges data you need. The app only calls Stripe GET APIs. Configure charges, recurring billing, refunds and cancellations in Stripe's own dashboard.

3. For local work, put the key in `admin/.env.local` as `STRIPE_SECRET_KEY`, with `STRIPE_MODE=test`. For hosted work, put it in the agency Worker secrets.

4. Open a Stripe customer and copy its `cus_...` ID. In the admin client form, paste it into **Stripe customer ID (test)**. Live IDs have a separate field.

5. Restart after adding environment keys. Click **Payments > Sync Stripe**.

6. Verify an existing test invoice and payment against Stripe. Check the amount, currency, customer and payment/refund status. A paid invoice can include credits or an out-of-band payment, so the Transactions view records actual charges.

Import fetches the latest 100 invoices/charges plus up to two older pages per run for each linked customer. Subsequent syncs continue older history, then rotate through it again for reconciliation. Newest records refresh every run. The hosted Worker also syncs hourly at minute 17. History may take several runs for a large customer. Never combine test/live records or different currencies into one total.

Only customers explicitly linked to clients are imported. One-time charges without a Stripe customer are not assigned by guessed email matching. Attach the correct customer in your billing process. Pending/unpaid invoices need an actual due date to appear overdue; a service agreement alone does not invent an unpaid invoice.

Stripe webhooks

For hosted use, send webhooks directly to:

```
https://YOUR-AGENCY-WORKER.workers.dev/api/webhooks/stripe
```

This route verifies Stripe signatures and does not require the agency bearer token. Keep it outside the Cloudflare Access application protecting your admin UI. Use API/event version **2025-09-30.clover** to match the pinned reader.

Select these events:

```
invoice.created
invoice.updated
invoice.paid
invoice.payment_failed
invoice.voided
invoice.marked_uncollectible
charge.succeeded
charge.updated
charge.refunded
customer.subscription.created
customer.subscription.updated
customer.subscription.deleted
```

Store the endpoint's `whsec_...` signing secret as `STRIPE_WEBHOOK_SECRET` on the Worker. Repeated events are safe; the handler fetches current Stripe objects so out-of-order events cannot restore an old payment state. Scheduled sync recovers missed updates. Disputed charges are also identified during synchronization.

For local testing with the official Stripe CLI:

```
stripe listen --forward-to http://127.0.0.1:3001/api/webhooks/stripe
```

Use the CLI's signing secret locally. Test mode and live mode have different keys, customer mappings and webhook secrets. Switch them together and verify in Stripe before relying on live records. This app does not create charges or shut websites down when a payment fails.

4. Enable website health

For local manual checks, ensure `MONITOR_ALLOWED_HOSTS` in `admin/.env.local` contains the exact public hostname being checked. Restart after changing it.

1. Edit the website record and confirm its deployed HTTPS URL.

2. Set expected page text that identifies the website, such as `Breeze`.

3. Enable monitoring and save.

4. Open **Websites > Health & incidents** and click **Run checks**.

Checks reject redirects. Use the final URL, such as a `www` hostname if that is where the site redirects. Add that exact hostname to the approved list. Private IP addresses, credentials in URLs, HTTP and custom ports are rejected.

The deployed Worker checks enabled sites every five minutes. Local development does not run unattended checks; click Run checks or run `npm run monitor` while working. The UI refreshes records every minute. Results older than 20 minutes are stale. Two consecutive failures open an incident; a healthy result resolves it. Incident notices are visible in the admin. Email/SMS/push alerts are not enabled. Raw checks are retained for 90 days, with incident history retained separately.

Connect Breeze CMS diagnostics

The code adds `/api/monitor/status` to Breeze's CMS Worker. It is disabled until a separate monitoring token is configured and must be deployed before use.

1. Generate a new random token of at least 32 characters. Keep it separate from `CMS_WRITE_TOKEN`; the code rejects reusing the write token.

2. From `breeze-web/workers/cms`, set the live monitoring secret:

```
npx wrangler secret put CMS_MONITOR_TOKEN --config wrangler.live.jsonc
```

3. Deploy the reviewed Breeze Worker code using your existing live procedure and exact existing resource bindings. This feature needs no database migration.

```
npx wrangler deploy --config wrangler.live.jsonc
```

4. Set `CMS_MONITOR_TOKENS_JSON` in the agency's server secrets to:

```
{"breeze":"PASTE-THE-SEPARATE-READ-ONLY-TOKEN"}
```

5. Confirm the website's CMS status URL is `https://breeze-live-api.nikodola.workers.dev/api/monitor/status`, and approve that hostname in the agency monitor's allowed-host list.

6. Run checks. The response checks database/media readiness and pending publication delivery. It exposes no drafts, content documents or write capabilities.

For another CMS adapter, return JSON with `version:1`, the registered `siteId`, `environment:"live"`, `ready:boolean`, `pendingPublications:integer`, and optional `oldestPendingAt/lastSuccessfulDeliveryAt`. Require a separate server token.

5. Prepare the cloud agency backend

All commands in this section run from `admin/workers/api` unless stated otherwise. Create a NEW agency database. Never reuse Breeze's live or test databases. If you already deployed the agency backend, apply the new migrations with `npm run migrate:remote` before deploying the updated Worker. Migration 0002 adds Communications and North Macedonia/USD defaults. Existing service prices and their currencies are preserved. Local SQLite upgrades automatically once.

```
npm install
npx wrangler login
npx wrangler d1 create website-products-admin
```

Copy the returned database ID into `wrangler.jsonc` in place of `REPLACE_WITH_YOUR_NEW_AGENCY_DATABASE_ID`. Confirm your account and database name. Then apply the agency migration:

```
npm run migrate:remote
```

Generate your own 32+ character platform token, for example:

```
node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
```

Store it securely; do not paste it into source code. Set Worker secrets using the interactive prompts:

```
npx wrangler secret put PLATFORM_API_TOKEN
npx wrangler secret put STRIPE_SECRET_KEY
npx wrangler secret put STRIPE_WEBHOOK_SECRET
npx wrangler secret put CMS_MONITOR_TOKENS_JSON
```

You may omit Stripe/CMS secrets until ready. Set `STRIPE_MODE` and approved monitor hosts in `wrangler.jsonc`, then:

```
npm run build
npm run deploy
```

The build is a dry run; deploy is the explicit cloud write. Copy the returned Worker origin for `PLATFORM_API_URL`. Cron schedules are configured in the file. Select a Cloudflare plan with sufficient D1 queries and Worker limits for your record volume; large imports perform many reads/writes.

6. Deploy the private admin

1. Keep this admin source in its own Git repository, preserving Breeze's existing repository/history. Do not commit `.env.local`, `.data`, exports or backups.

2. In Vercel, import the admin repository. The Next.js app directory is `admin` if the repository contains the workspace, or `.` if only admin source is imported. Include `shared/` and `workers/api/migrations/` with the admin source.

3. Use Node 22, the normal Next.js preset and `npm run build`.

4. Set Vercel environment variables: `PLATFORM_API_URL`, matching `PLATFORM_API_TOKEN`, `ADMIN_ORIGIN`, `CF_ACCESS_TEAM_DOMAIN`, `CF_ACCESS_AUD`, and `ADMIN_ALLOWED_EMAILS`. Use the exact final HTTPS custom origin for `ADMIN_ORIGIN`, with no slash at the end.

5. Create a Cloudflare Access self-hosted application for your admin custom domain. Route that domain through Cloudflare to the Vercel project using the exact DNS target Vercel provides. Configure an identity provider and allow only your owner email. Require MFA in the Access/identity-provider policy.

6. Copy the application's audience (AUD) and team hostname into Vercel. The app verifies Access JWT signatures, issuer, audience, expiry, subject and owner email. Signing in with another account does not grant access.

7. Redeploy and open the protected custom domain. Verify your authorized login. A direct `.vercel.app` URL without an Access token must not expose records.

Production fails closed when owner identity or the persistent backend is missing. The local shortcut is ignored in production, including if its flag is set true. Do not attempt to use Vercel's local filesystem as the business database.

7. Move local records to a fresh cloud database

Create a local JSON export before changing your local backend settings:

```
npm --prefix admin run db:export
```

The command prints the private file path under `backups/`. Then set `PLATFORM_API_URL` and `PLATFORM_API_TOKEN` in `admin/.env.local` to your new backend. From the workspace root:

```
npm --prefix admin run db:push -- "C:\FULL\PATH\TO\platform-export.json"
```

Import only accepts a fresh agency database containing the default Breeze seed. It refuses to overwrite an active business database. Records are imported as an atomic batch, with an audit entry. Restart the admin and compare clients, sites, services and tasks. Keep the local database/export until you verify the result.

For a large database exceeding request/batch limits, use a planned D1 import with a database administrator rather than repeatedly posting the file.

8. Add another website

From the workspace root, preview the intended folder and port:

```
npm run site:new -- --id client-two --name "Client Two" --dry-run
```

Create the project by omitting `--dry-run`. Add `--register` while the local admin is running to create its client/site records. Use `--client EXISTING-CLIENT-ID` to attach it to an existing client. Registration uses a placeholder URL with monitoring disabled; update that URL after deployment.

```
npm run site:new -- --id client-two --name "Client Two" --register
npm --prefix client-two-web install
npm --prefix client-two-web run dev
```

The script refuses existing folders, validates IDs, reserves ports 3000/3001, and selects a free port based on site manifests. Each new site has its own CSS, content, app, public assets and non-secret `site.config.json`.

The clean starter uses file content. It includes no Breeze customer content, credentials or visual CMS. Provision a site's own CMS/schema/storage when that client needs visual editing. Monitoring is independent of CMS capabilities. Give every site its own hosting project, domains and secrets. Run its build and verify the client's actual form delivery before launch.

9. Backups and restoration

For a local SQLite backup from the workspace root:

```
npm run backup
```

This creates a consistent snapshot under `backups/`, even with WAL enabled. Keep an additional private copy outside this computer. The admin also offers **Settings > Export records** and payment CSV exports. JSON preserves all stored business records; CSV amounts are integer minor units and include currency.

Restore a JSON export to a NEW local file:

```
npm --prefix admin run db:import -- "C:\PATH\export.json" "C:\PATH\restored.sqlite"
```

The restore refuses to overwrite existing files and checks database integrity. Set `PLATFORM_DB_PATH` in `admin/.env.local` to the new path, clear the cloud URL if testing locally, and restart. A full SQLite snapshot can also be opened by pointing `PLATFORM_DB_PATH` at a copy of that snapshot.

For cloud D1 backup, run from `admin/workers/api`:

```
npx wrangler d1 export website-products-admin --remote --output ../../../backups/agency-cloud.sql
```

Create the backups directory first if needed. Restore into a NEW recovery D1 database and verify it before switching the Worker's binding. Keep your configured secrets separately. Agency backups do not include Breeze's D1/R2 media; maintain and test those backups independently.

10. Checks and known limits

From the root, `npm run check` runs admin types, unit tests and its production build. After installing Worker dependencies, also run:

```
npm --prefix admin run test:cloud
npm --prefix admin/workers/api run build
npm --prefix breeze-web run cms:check
```

Implementation verification: admin unit tests, local D1 integration, Worker dry-run bundle and admin build pass. Breeze's new monitoring checks pass. The existing Breeze suite has three unrelated stale baseline assertions (old editor labels and the former pre-Housecall-Pro form); see `docs/VERIFICATION.md`. The Breeze build can be checked with CMS disabled when its local Worker is off.

You requested to inspect the dashboard yourself; visual dashboard QA is left to you. Live Stripe, owner login, cloud deployment and live monitor integrations require your keys/accounts and were not exercised against production.

The dashboard displays up to 1,000 records per business list, 500 recent raw checks and 100 incidents/activity items; exports contain all stored business rows. The monitor handles up to 100 enabled sites per run. This is a practical agency foundation; add server pagination and larger job queues as volume grows.

Official references

- Stripe invoice states: https://docs.stripe.com/invoicing/overview

- Stripe webhooks: https://docs.stripe.com/webhooks

- Cloudflare Access JWT validation: https://developers.cloudflare.com/cloudflare-one/access-controls/applications/http-apps/authorization-cookie/validating-json/

- Cloudflare scheduled handlers: https://developers.cloudflare.com/workers/runtime-apis/handlers/scheduled/

- Cloudflare D1 migrations: https://developers.cloudflare.com/d1/reference/migrations/

- Vercel project configuration: https://vercel.com/docs/projects/overview